Python for Web Development

Lesson 4: Form Handling and Validation

Form Handling and Validation - Study Notes

Key Concepts

1. **Processing POST data** – retrieving, sanitizing, and using form data sent via HTTP POST.
2. **WTForms integration** – defining form classes, fields, and built in validators.
3. **Custom validation** – writing reusable validator functions and methods.
4. **CSRF protection** – preventing cross site request forgery with Flask WTF's CSRF token.
5. **Error handling & feedback** – displaying validation errors back to the user.

Summary

Handling user input securely is a cornerstone of web development. In Flask, a POST request carries the form payload in `request.form`. While you can manually extract values, using **WTForms** (often via **Flask WTF**) gives you a declarative, reusable way to describe fields, apply validation rules, and automatically generate HTML widgets.

WTForms also ships with a **CSRF token** mechanism. When you enable `FlaskForm` (or `Form` with `csrf=True`), a hidden field containing a signed token is added to every rendered form. On submission, Flask WTF verifies the token, ensuring the request originated from a page you served, which mitigates CSRF attacks.

By combining POST processing, WTForms validation, and CSRF protection, you can build robust, user friendly forms that gracefully handle errors and keep your application safe from common web threats.

Important Points

- **Accessing POST data**:

```python
from flask import request
name = request.form.get('name')        # returns None if missing
email = request.form['email']          # raises BadRequestKeyError if missing
```

- **Creating a WTForm**:

```python
from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import DataRequired, Email, Length, EqualTo
```

• **Built in validators** (most common):
- `DataRequired()` – field cannot be empty.
- `Email()` – checks for a valid email address.
- `Length(min, max)` – enforces string length.
- `EqualTo('other_field')` – useful for password confirmation.
• **Custom validator example**:
```python
def validate_username(form, field):
    if User.query.filter_by(username=field.data).first():
        raise ValidationError('Username already taken.')
```
• **CSRF token**:
- Enabled automatically when you subclass `FlaskForm`.
- Requires `app.config['SECRET_KEY']` (or `WTF_CSRF_SECRET_KEY`).
- Render token in template with `{{ form.csrf_token }}`.
• **Displaying errors** (Jinja2):
```html
{% for field in form %}
    {{ field.label }} {{ field() }}
    {% for error in field.errors %}
        <span class="error">{{ error }}</span>
    {% endfor %}
{% endfor %}
```
• **Typical request flow**:
1. `GET /register` !’ instantiate empty form, render template.
2. `POST /register` !’ form receives `request.form`, `form.validate_on_submit()` runs validators + CSRF check.
3. If valid !’ process data (e.g., create user, flash success, redirect).
4. If invalid !’ re render template with populated form and error messages.

Examples

Example 1 – Simple Contact Form

Python (app.py)
```python
from flask import Flask, render_template, redirect, url_for, flash
from flask_wtf import FlaskForm
from wtforms import StringField, TextAreaField, SubmitField
from wtforms.validators import DataRequired, Email, Length
app = Flask(__name__)
app.config['SECRET_KEY'] = 'a  very  secret  key'   # needed for CSRF
```

```
class ContactForm(FlaskForm):
    name = StringField('Name', validators=[DataRequired(), Length(max=50)])
    email = StringField('Email', validators=[DataRequired(), Email()])
    message = TextAreaField('Message', validators=[DataRequired(), Length(max=500)])
    submit = SubmitField('Send')
@app.route('/contact', methods=['GET', 'POST'])
def contact():
    form = ContactForm()
    if form.validate_on_submit():          # POST + CSRF + field validation
        # Here you would normally send an email or store the message
        flash('Your message has been sent!', 'success')
        return redirect(url_for('contact'))
    return render_template('contact.html', form=form)
```

Template (contact.html)

```html
<!doctype html>
<title>Contact</title>
<h1>Contact Us</h1>
<form method="POST">
    {{ form.hidden_tag() }}   {# renders CSRF token and any hidden fields #}
    <p>
        {{ form.name.label }}<br>
        {{ form.name(size=32) }}<br>
        {% for err in form.name.errors %}
            <span class="error">{{ err }}</span>
        {% endfor %}
    </p>
    <p>
        {{ form.email.label }}<br>
        {{ form.email(size=32) }}<br>
        {% for err in form.email.errors %}
            <span class="error">{{ err }}</span>
        {% endfor %}
    </p>
    <p>
        {{ form.message.label }}<br>
        {{ form.message(rows=5, cols=40) }}<br>
        {% for err in form.message.errors %}
            <span class="error">{{ err }}</span>
        {% endfor %}
    </p>
```

```
    <p>{{ form.submit() }}</p>
</form>
```

Explanation: The `ContactForm` defines three fields with appropriate validators. `form.validate_on_submit()` handles both the HTTP method check and CSRF verification. Errors are displayed next to each field, and a flash message confirms success.

Example 2 – Registration Form with Custom Username Check

Python (models.py) (simplified user model)

```python
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(30), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(128), nullable=False)
```

Python (forms.py)

```python
from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import DataRequired, Email, Length, EqualTo, ValidationError
from models import User
class RegistrationForm(FlaskForm):
    username = StringField('Username',
                           validators=[DataRequired(), Length(min=3, max=30)])
    email = StringField('Email',
                        validators=[DataRequired(), Email(), Length(max=120)])
    password = PasswordField('Password',
                             validators=[DataRequired(), Length(min=6)])
    confirm = PasswordField('Confirm Password',
                            validators=[DataRequired(),
                                        EqualTo('password', message='Passwords must match')])
    submit = SubmitField('Register')
    # ----- custom validator method -----
    def validate_username(self, field):
        if User.query.filter_by(username=field.data).first():
            raise ValidationError('Username already exists.')
    def validate_email(self, field):
        if User.query.filter_by(email=field.data).first():
            raise ValidationError('Email already registered.')
```

```
**Python (routes.py)**
```python
@app.route('/register', methods=['GET', 'POST'])
def register():
    form = RegistrationForm()
    if form.validate_on_submit():
        # Password hashing omitted for brevity
        new_user = User(username=form.username.data,
                        email=form.email.data,
                        password_hash=form.password.data)
        db.session.add(new_user)
        db.session.commit()
        flash('Registration successful! You can now log in.', 'success')
        return redirect(url_for('login'))
    return render_template('register.html', form=form)
```
Explanation:
• The form class contains **method style custom validators** (`validate_<fieldname>`) that automatically run after built in validators.
• Because the form inherits from `FlaskForm`, a hidden CSRF token is added (`{{ form.hidden_tag() }}` in the template).
• On a successful POST, the user is persisted to the database; otherwise, errors are shown inline.

Quick Review
1. **What does `form.validate_on_submit()` check internally?**
2. **Name two built in WTForms validators and describe when you would use each.**
3. **How does Flask WTF protect against CSRF attacks?**
4. **Write a short snippet that adds a custom validator to ensure an integer field is even.**
5. **If a form field fails validation, how can you display its error messages in a Jinja2 template?**

Further Reading
• Flask WTF documentation – deeper dive into CSRF configuration and file uploads.
• WTForms Advanced Validators – `Regexp`, `NumberRange`, `AnyOf`, `NoneOf`.
• Flask Login integration – tying validated login forms to user session management.
• Security best practices – rate limiting, reCAPTCHA, and protecting against XSS in form inputs.
• Asynchronous form handling with AJAX and Flask RESTful APIs.
```